2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW) | 978-1-6654-7679-9/22/$31.00 ©2022 IEEE | DOI: 10.1109/ISSREW55968.2022.00050

A Stochastic Petri net Model of Continuous Integration and Continuous Delivery

Sushovan Bhadra
Department of Electrical and Computer Engineering
University of Massachusetts Dartmouth
Dartmouth, Massachusetts, USA
sbhadra@umassd.edu

***Abstract*— Modern software development organizations rely on continuous integration and continuous delivery (CI/CD), since it allows developers to continuously integrate their code in a single shared repository and automates the delivery process of the product to the user. While modern software practices improve the performance of the software life cycle, they also increase the complexity of this process. Past studies make improvements to the performance of the CI/CD pipeline. However, there are fewer formal models to quantitatively guide process and product quality improvement or characterize how automated and human activities compose and interact asynchronously. Therefore, this paper develops a Petri net model to analyze a CI/CD pipeline to improve process performance in terms of the probability of successfully delivering new or updated functionality by a specified deadline. The utility of the model is demonstrated through a sensitivity analysis to identify stages of the pipeline where improvements would most significantly improve the probability of timely product delivery.**

Index Terms—CI/CD pipeline, petri net, stochastic petri net, performance, sensitivity analysis

I. Introduction

DevOps [1] is a modern practice that combines software development and operations to accelerate the delivery of new software capabilities while preserving high-quality. A continuous integration and continuous development (CI/CD) [2] pipeline forms the backbone of modern-day DevOps operations, providing a framework for automation that promotes problem identification and resolution. Few past CI/CD studies attempt to formally model DevOps-specific activities. However, they do not provide direct insight into the overall benefits achieved by modeling a CI/CD pipeline. Without a comprehensive pipeline model that constitutes an organization's DevOps practices, it will be difficult to explicitly characterize how automation and human activities are composed and interact asynchronously to maximize process performance. A generalized CI/CD pipeline model is therefore needed to measure the potential effectiveness of process improvements and guide the allocation of limited resources to such process improvement efforts.

A. Research Questions

CI/CD techniques are becoming popular in software development companies and have enjoyed widespread adoption in recent years. However, for these techniques to achieve their full potential, quantitative process modeling and optimization are needed. Modeling a full CI/CD pipeline can help stakeholders determine modifications and implementation of research advances that are of greatest benefit. Toward this end, we seek to answer the following research questions.

- *What are the most common set of activities found in large-scale real-world CI/CD pipelines?* This information will be used to develop rich and realistic models of CI/CD pipelines employed by industry. Information contained in open source repositories and related publications in the open literature will inform the creation of enhanced models.
- *How can the wide variety of research on artificial intelligence and machine learning for software engineering contribute to improving the CI/CD pipeline?* Many techniques have been proposed to improve CI/CD performance and other process and product attributes, but they impact on pipeline improvement is rarely reported.
- *How can we optimize process and product metrics relevant to the CI/CD pipeline to achieve stakeholder needs?* Understanding the cost to improve process and product metrics like delivery time, down time, reliability, security, and safety will judiciously guide allocation of resources to achieve the desired process improvement activities.

B. Related work

One line of research has focused on the use of machine learning (ML) techniques to enhance process performance. For example, Robinson et al. [3] implemented machine learning techniques that combines data retrieval, imputation, preprocessing, and training to support continuous experimentation and development. Nogueira et al. [4] employed machine learning techniques to enable quick feedback and earlier decision making in the software development process of a CI/CD pipeline.

Another emerging line of research applies different modeling languages and frameworks to the CI/CD pipeline in order to improve performance or security at different stages. For example, Khanan et al. [5] presented an explainable Just-In-Time defect prediction framework to automatically generate feedback to developers, indicating the riskiness of each commit, explaining why such a commit is risky, and suggesting risk mitigation plans. Kruglov et al. [6] proposed a method to recognize energy consumption bottlenecks and provide recommendations to improve the structure and run-time behavior of software. Sachdeva [7] proposed a cloud-based environment to build and test application software before deploying to physical environments, reducing the time wasted

978-1-6654-7679-9/22/$31.00 ©2022 IEEE
DOI 10.1109/ISSREW55968.2022.00050

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:04:18 UTC from IEEE Xplore. Restrictions apply.

in manual test execution. Liao et al. [8] designed a queuing theoretic CI/CD model and an agent-based simulation to analyze the pipeline under different settings and operating scenarios to improve economic outcomes, while Medvedev and Aksyonov [9] applied multi-agent simulation to test different settings and scenarios of a CI/CD pipeline to simplify management, improve economic outcomes, and achieve high efficiency. Tegeler et al. [10] used a graphical modeling language to formally model different tasks and their dependencies to assess development practices and productivity. Chen et al. [11] presented a framework to assess the tradeoff between process and product metrics, including delivery time, down time, safety, and security. Paule et al. [12] employed threat analysis to detect vulnerabilities in two CI/CD pipelines to improve process security. Muñoz et al. [13] identified security gaps in a CI/CD pipeline and proposed a novel tool to ensure security assertions against attacks. While these past studies have employed distinct modeling languages, frameworks and machine learning techniques for process improvement in different CI/CD stages, a comprehensive model of the CI/CD pipeline is needed to establish baseline measurements of process effectiveness and efficiency as well as guide process improvement efforts.

To overcome the shortcomings of past studies, this paper presents a Stochastic Petri net (SPN) [14] model capable of characterizing a complete CI/CD pipeline to perform upfront analysis prior to implementation or modification.

II. Proposed Approach

A. CI/CD pipeline

CI/CD is a set of processes [2] to automate software development stages to accelerate the delivery of high-quality products to customers according to the new features they request. Although different software process configurations exist, the common set of activities found in CI/CD pipelines is shown in Figure 1.

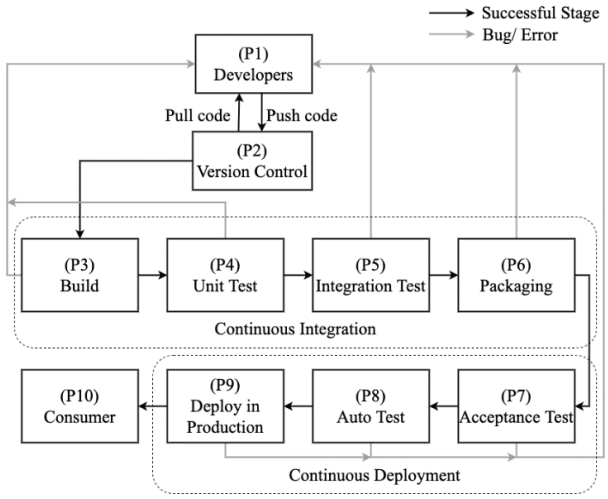

Figure 1-CI/CD Pipeline

Figure 1 represents the CI/CD pipeline steps, starting with state P1, where developers work independently creating their own coding branch. Next, each developer pushes their codes to the version control where developers share code in a single repository, which allows them to make changes, check resources, and updates that are managed by the task manager in the state P2. An example of a modern version control system is GitHub, where developers work in parallel without worrying about overwriting the code of others.

Continuous integration consists of states P3 through P6. A build server attempts to compile code and execute tests, producing software quality metrics [2] such as code coverage in state (P3). The CI server monitors the developer's repository and starts the build process upon each check-in. Unit tests (P4) test various functions to ensure they produce expected results. Failure in the build state indicates something is missing or wrong in the code development state. Developers are notified of test results for review, where they pull the code from the repository state (P2), make the changes, and return it to version control to be recompiled, built, and tested again. Even if the unit test is successful, it does not guarantee reliable software, since the software is tested unit by unit in that state. After passing unit testing, the code proceeds to integration testing (P5), which tests the reliability and performance of the entire system. If there is an error in an integration test, the code is sent back to the developer state (P1). If the code passes the integration test, developers are notified for approval prior to software packaging. After successful integration testing and developer approval, the code is compiled to a file format in the packaging state (P6). Failure in during packaging returns to the developers (P1). Some popular CI servers include Jenkins, Bamboo, Team foundation server, and Cruise-control. Successful integration leads to the continuous deployment state.

Continuous Deployment is carried out between states P7 and P9, beginning with acceptance testing (P7), which is performed manually by a review team that provides feedback about the level of acceptability from the user's perspective. Auto test starts in state (P8), which observes the performance and reliability of the software in the working environment shared with a limited number of customers. If these customers encounter difficulties working with the application, it is checked again by the developers. These checks are manual and may therefore be slower. After successfully passing these tests, the software is deployed to production in state (P9). Any failure in the production state requires attention from the developers (P1). Finally, the software is released to the consumer state (P10) such as an app store.

B. Petri net modeling

In this section petri net [15] modeling is explained to model the CI/CD pipeline. Petri nets are a mathematical modeling language technique which is used to model discreet event distributed system. For modeling purpose, we created a stochastic petri net model. Figure 2 illustrates the SPN model of the CI/CD pipeline introduced in Figure 1.

Identify applicable funding agency here. If none, delete this text box.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:04:18 UTC from IEEE Xplore. Restrictions apply.

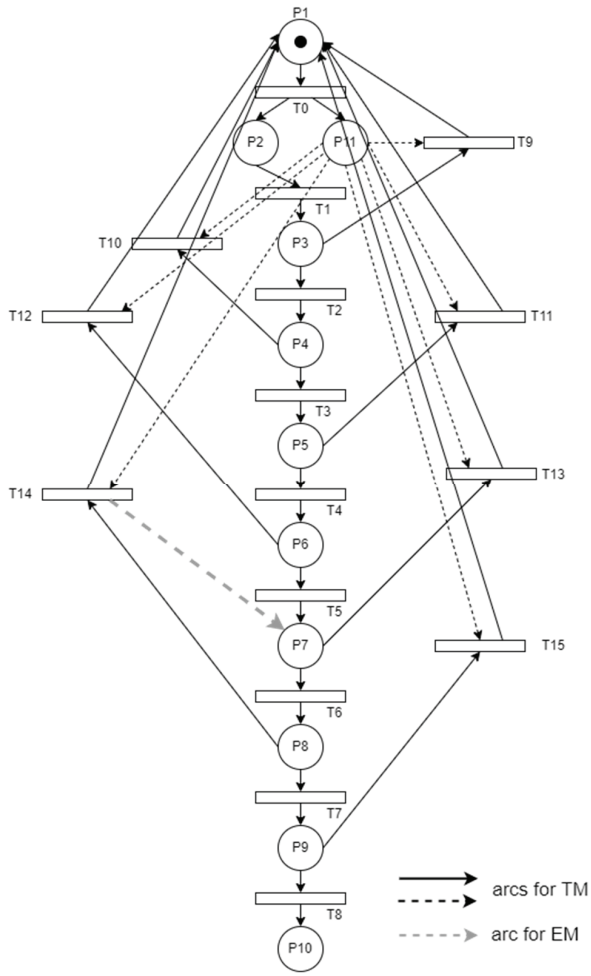

Figure 2-SPN model of CI/CD pipeline

In Figure 2, places P1 to P10 represent the same stages of the CI/CD pipeline introduced in Figure 1. Transitions T0 through T8 indicate successful forward transitions, while T9 to T15 represent feedback transitions. The token in place P1 indicates code to be tested in different CI/CD stages prior to release. When transition T0 fires, the token moves to the version control states (P2 and P11), where P11 holds the updated version of the code to be pulled by developers any time the code needs to be updated or fixed. As a result, the initial marking M1(1,0,0,0,0,0,0,0,0,0,0) changes to marking M2(0,1,0,0,0,0,0,0,0,0,1). Hence, P11 is connected to all of the error transition branches, which are marked with black dashed lines. Tokens in places P2 and P11 indicate that the code is ready to be tested. Tokens will be removed from P2 by firing transition T1 and creating one token in place P3 for build testing, expressed as marking M3(0,0,1,0,0,0,0,0,0,0,1). In this marking, both transitions T2 and T9 are enabled to fire. A successful test allows T2 to fire for the unit test at place P4, but failure triggers T9, which return the token to the initial place P1 with corresponding marking M1. Firing transition T2 leads to unit testing in place P4 possessing marking M4(0,0,0,1,0,0,0,0,0,0,1). As discussed in Section A, the process continues until reaching the final stage at place P10. An enhanced CI/CD model (EM) is shown using a gray dashed line in Figure 2 from transition T14 to place P7. Transition T14 indicates errors in auto test state in place P8 and P7 represents the acceptance testing stage, where reviewers can provide limited help to solve customer problems.

C. Performance analysis

This section compares the performance of the traditional and enhanced CI/CD pipelines, using the SPN model to quantitatively assess the probability of successful delivery by a specific time. All experiments assumed a product deadline of 150 time units after the start of the simulation for the product to be released to the customer and the simulations were carried out with the Mercury Tool [16]. More generally, Figure 3 shows the probability of successful delivery as a function of time under the traditional and enhanced models.

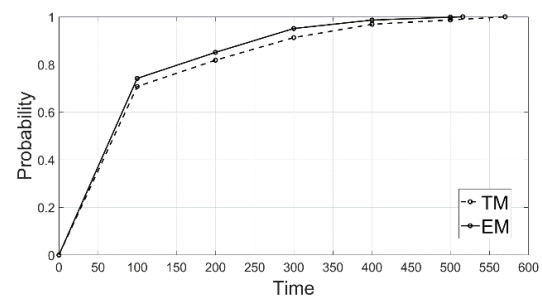

Figure 3- Probability of successful produce delivery under the traditional and enhanced CI/CD models

Figure 3 indicates that the enhanced model improves the probability of successful product delivery by time 150 from 77.4% to 80.02%. We also simulated the two SPN models for a greater time period to determine which approach would complete the delivery process faster. Figure 3 also shows that the enhanced model delivers the product with 100% probability around time 516, whereas the traditional model requires 570 time units, which means that the enhanced model reduces the delivery time by nearly 10% (516/570=0.905). In other words, when reviewers are able to resolve some problems without developers, the delivery time can be reduced, substantially improving CI/CD performance.

The simulation provides the transition firing rate log, where there is no significant difference between all the transitions, except T6 increased 27.5% (7.64/5.99), T7 increased 40.8%, and T8 increased 18.4% indicating resolution of problems substantially increased forward progress, thereby improving delivery time.

D. Sensitivity analysis

In the second experiment, we assessed the impact of decreasing the failure rate in each state associated with testing, which lowers the rate of feedback transitions that require rework. For each rate, the failure rate was halved while holding all other transition rates are constant.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:04:18 UTC from IEEE Xplore. Restrictions apply.

Table 1-Impact of failure rate reduction on probability of successful product delivery under the traditional CI/CD pipeline

Transition	Prob. (%)	Transition	Prob. (%)
T9=0.50	80.30	T13=1.00	82.41
T10=0.02	80.44	T14=0.85	82.89
T11=0.15	80.72	T15=0.68	84.10
T12=1.86	81.98		

Table 1 indicates that reducing the failure rate in later stages such as auto test and deploy in production, corresponding to the transitions T14 and T15 respectively, increase the probability of successful delivery more because it takes additional time to reach these later stages and therefore would require more time overall, if problems were not encountered until these stages.

E. Milestones

The proposed research includes the following timelines: 2022: Generalize the model and enhanced the accuracy of analysis with statistical data from large real-world CI/CD projects available in open source repositories. 2023: Review and critically analyze emerging research on AI for software engineering and map the implications of these studies to the CI/CD model. 2024: Develop multi-objective optimization procedures based on the model to cost effectively enhance the attributes of the CI/CD pipeline based on stakeholder needs. 2025: Publication preparation and dissertation defense.

III. Expedted Contributions

This paper presented a CI/CD pipeline model using stochastic Petri net techniques to evaluate the probability of successfully delivering a software product or update by a specified deadline. The illustrations demonstrated that Petri net modeling was capable of characterizing a complete CI/CD pipeline to (i) perform upfront analysis prior to implementation, and (ii) compare different pipelines. Two models of the CI/CD pipeline were considered for illustration: a traditional model and an enhanced model. Illustrations showed that the enhanced CI/CD approach improved the probability of successful delivery by nearly 10% over the traditional approach. Further experiments performed sensitivity analysis in the context of the enhanced CI/CD model to quantify how reductions in failure rates of various transitions corresponding to stages of testing could improve the probability of successful product delivery. These results indicated that the Petri net modeling paradigm can also be used to identify pipeline improvements that will most effectively improve the software process.

Future research will remove the assumption of exponentially distributed transition parameters to achieve greater realism and enable more accurate analysis. The accuracy of these models will also be assessed in the context of various classes of software systems of differing size and complexity.

Acknowledgment

This material is based upon work supported by the Office of Naval Research under Award Number N00014-22-1-2012. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the Office of Naval Research.

References

[1] P. Jha, and R. Khan, "A review paper on DevOps: Beginning and more to know." *International Journal of Computer Applications* 180.48 (2018): pp. 16-20.

[2] S. Rossel, *Continuous Integration, delivery and Deployment*, 1st ed. Birmingham-Mumbai: Packet Publishing Ltd, October 2017.

[3] K. Robinson, C. Billman, M. Masih, K. Rose, X. Wang and K. Hundman, "A Full-Stack Machine Learning Environment for Rapidly Evolving Industry Applications," *Proc. IEEE International Conference on Data Science and Advanced Analytics*, 2021, pp. 1-3.

[4] A.F. Nogueira, J. C.B Ribeiro, M.A Zenha-Rela and A. Craske, "Improving la redoute's ci/cd pipeline and devops processes by applying machine learning techniques," *Proc. International Conference on the Quality of Information and communication Technology*, 2018, pp. 282-286.

[5] C. Khanan, W. Luewichana, K. Pruktharathikoon, J. Jiarpakdee, C. Tantithamthavorn, M. Choetkiertikul, C.Ragkhiwetagul, T. Sunetnanta, "JITBot: An Explainable Just-In-Time Defect Prediction Bot." *Proc. IEEE/ACM International Conference on Automated Software Engineering*. 2020.

[6] A. Kruglov, G. Succi, and X. Vasuez. "Incorporating energy efficiency measurement into CI\CD pipeline." *Proc. 2nd European Symposium on Software Engineering, pp. 14-20. 2021.*

[7] R. Sachdeva, "Automated testing in DevOps." Proc. Pacific Northwest Software Quality Conference. 2016.

[8] Q. Liao, "Modelling CI/CD Pipeline Through Agent-Based Simulation," *IEEE International Symposium on Software Reliability Engineering Workshops*, 2020, pp. 155-156.

[9] D. Medvedev, and K. Aksyonov. "The Development of a Simulation Model for Assessing the CI/CD Pipeline Quality in the Development of Information Systems Based on a Multi-Agent Approach." *MATEC Web of Conferences. Vol. 346. EDP Sciences, 2021*

[10] T. Tegeler, F. Gossen, and B. Steffen, "A model-driven approach to continuous practices for modern cloud-based web applications." *International Conference on Cloud Computing, Data Science & Engineering*. 2019.

[11] K. Chen, Z. Faddi, V. Nagaraju, and L. Fiondella, "Quantifying the Impact of Staged Rollout Policies on Software Process and Product Metrics." *Proc. Annual Reliability and Maintainability Symposium*, Tucson, AZ, Jan 2022.

[12] C. Paule, T. F. Düllmann, and A. V. Hoorn. "Vulnerabilities in continuous delivery pipelines? A case study." IEEE international conference on software architecture companion (ICSA-C), pp. 102-108. 2019.

[13] A. Muñoz, A. Farao, J.R.C. Correia, and C. Xenakis, "P2ISE: Preserving Project Integrity in CI/CD Based on Secure Elements." *Information*, vol. 12, no. 9, 2021: p 357.

[14] G. Ciardo, R. German and C. Lindemann, "A characterization of the stochastic process underlying a stochastic Petri net," *IEEE Transactions on Software Engineering*, vol. 20, no. 7, pp. 506-515, July 1994.

[15] T. Murata, "Petri nets: Properties, analysis and applications." *Proceedings of the IEEE*, Apr 1989, vol. 77, no. 4, pp.541-580.

[16] B. Silva, R. Matos, G. Callou, J. Figueiredo, D. Oliveira, J. Ferreira, J. Dantas, A. Lobo, V. Alves and P. Maciel, "Mercury: An integrated environment for performance and dependability evaluation of general systems." *Proc. Dependable Systems and Networks Conference*, 2015.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:04:18 UTC from IEEE Xplore. Restrictions apply.